面向对象程序设计报告

Mandol 复杂条件检索与多检索方式融合：系统分析与初步设计

项目	内容
课程	面向对象程序设计
研究方向	2.2 融合索引增强
研究对象	Mandol（AgentCombo/Mandol）
实验样本	LoCoMo10 的 `conv-26`（10 段长期对话中的一段）
日期	2026-10-22
姓名 / 学号	赵禹衡/2024K8009970007

摘要

Mandol 已经把历史对话整理成多层记忆，并用向量、BM25、SPLADE 和图关系提供了多条检索通道。但它的检索主链路回答的仍然是“多个后端的结果怎么合并”，查询里的实体、时间、关系、否定和状态更新没有被表示成一等条件，也就谈不到贯穿候选生成、融合与重排。

本文先给出 Mandol 的整体架构和检索层在其中的位置，再顺着代码梳理检索部分的文件结构、核心对象和执行流程，并说明仓库现有的测试与评测代码。之后用 exp0.1 阶段的五个探针（P4 到 P8）定位当前瓶颈：候选覆盖已经饱和，缺的是条件到证据的落地，以及 top-k 的排序精度。据此提出三层初步设计，即条件解析与证据 grounding（把查询条件落到具体记忆单元与对话上）、条件感知的融合与重排、端到端验证，并给出与原设计的差异、代码衔接方式、已否决的方案和停止条件。

---

1 研究背景与任务定位

1.1 Mandol 要解决的问题

Mandol 面向长期对话智能体。用户和助手的多轮对话先被整理成可存储、可关联、可更新的记忆，等到新问题到来时，系统再把真正有用的几条记忆交给下游大模型作答。按职责划分，这条链路分成三段：记忆组织负责把原始对话提炼成不同抽象层级的记

忆，记忆存储负责保存内容、元数据和各类索引，记忆检索负责按 query 找到相关记忆并排序。

本研究只处理第三段。记忆怎么分层、摘要怎么生成、图边怎么抽取，这些属于上游问题，本文不重新设计；本文关心的是给定一个已经建好的记忆库，怎样把问题里携带的条件用起来，把支撑答案的证据排到前面。

1.2 研究方向与三个研究问题

研究方向 2.2 的表述是：记忆访问可能同时包含语义相似、精确标识、属性约束和关系查询，单一访问方式或简单组合未必能支持复杂场景。落到 Mandol 上，这句话可以拆成几个具体问题：查询条件怎么组织成可比较的结构；BM25、dense、SPLADE 和图检索的候选怎么协调；新增能力怎么接进现有流程而不破坏兼容；检索效果、访问开销和维护成本又该怎么比较。

本文把研究问题写成三条：

1. RQ1（条件表示）：实体、时间、关系、否定和状态更新，如何从 query 解析成可检索、可比较的条件？
2. RQ2（多方式融合）：BM25、dense、SPLADE 以及 L0、hierarchical、graph、episodic 多塔，如何做条件感知的融合与重排？
3. RQ3（收益与成本）：条件感知融合相对原始 Mandol 能带来多少 Recall 与 nDCG 收益，代价是什么？

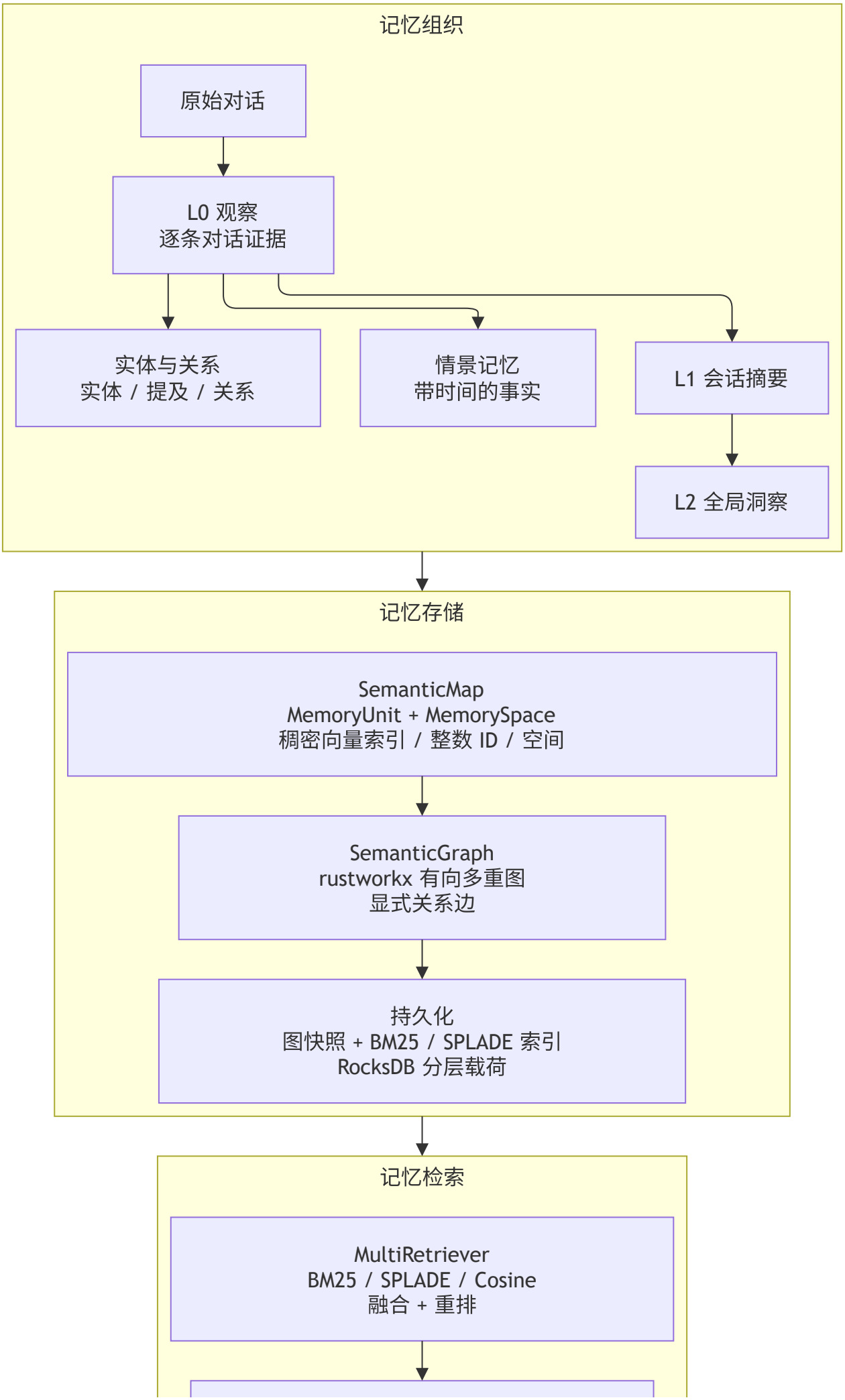
---

2 Mandol 的整体架构与检索层位置

2.1 三个高层次部分

部分	职责	关键概念
记忆组织	把原始对话提炼成不同抽象层级的记忆	L0 基础记忆、L1/L2 高层摘要、episodic 事件、entity 与 relation
记忆存储	保存记忆内容、元数据和索引	`MemoryUnit`、`MemorySpace`、`SemanticMap`、`SemanticGraph`
记忆检索	按 query 找到相关记忆并排序	多后端检索、空间过滤、RRF 融合、Cross-Encoder 重排、图扩展

三者的依赖是单向的：组织决定存储什么，存储决定检索能看到什么。检索层不产生新记忆，它只在已有单元里做筛选、融合和排序。


记忆组织
原始对话
L0 观察
逐条对话证据
实体与关系
实体 / 提及 / 关系
情景记忆
带时间的事实
L1 会话摘要
L2 全局洞察
记忆存储
SemanticMap
MemoryUnit + MemorySpace
稠密向量索引 / 整数 ID / 空间
SemanticGraph
rustworkx 有向多重图
显式关系边
持久化
图快照 + BM25 / SPLADE 索引
RocksDB 分层载荷
记忆检索
MultiRetriever
BM25 / SPLADE / Cosine
融合 + 重排

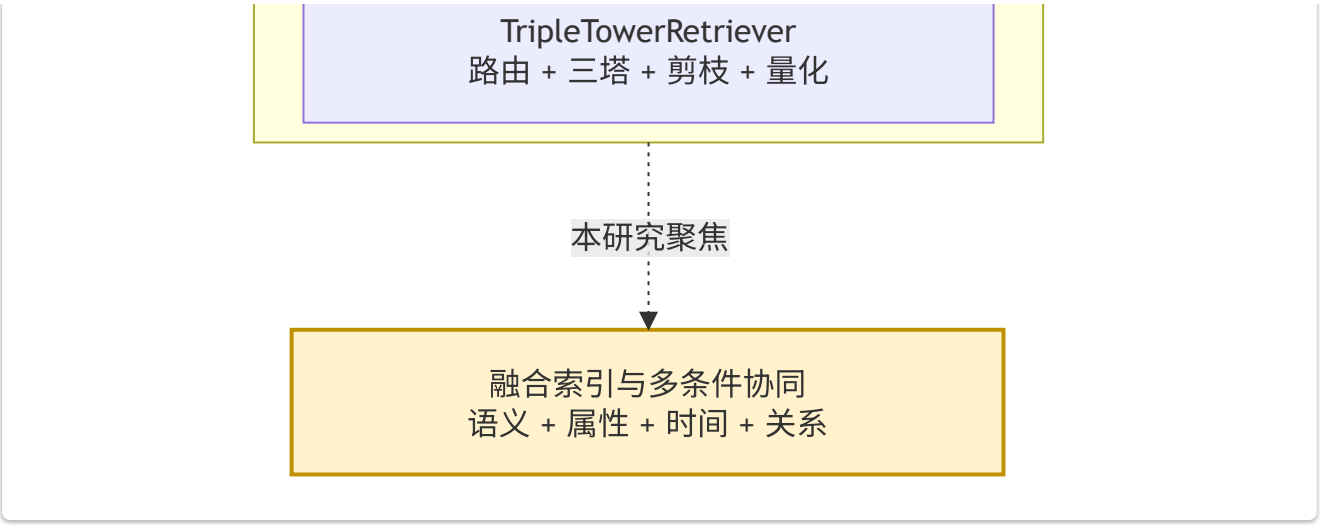


图 1　Mandol 的记忆组织、存储、检索三段，以及本研究落点

2.2 本研究的位置

本研究不重建记忆组织，也不改存储格式，改动集中在检索入口和检索内部三处。进入候选生成之前，先把 query 解析成条件；候选生成和融合阶段让条件参与打分；重排阶段按证据是否满足条件做选择。评测也要相应补到条件级，否则总体 QA 分数分不清是条件解析错了，还是排序错了。

---

3 检索部分的代码结构、核心对象与执行流程

3.1 代码结构

检索相关代码集中在 `src/mandol/retrieval/`，向上依赖 `core/` 的数据结构，向下被 `triple_retrieval/`、`memory_router/` 和 `benchmark_self_host/` 调用。按职责整理如下。

路径	作用
`src/mandol/core/memory_unit.py`	`MemoryUnit`：统一记忆
`src/mandol/core/memory_space.py`	`MemorySpace`：逻辑记忆
`src/mandol/core/memory_space_registry.py`	`TowerSpace`：检索塔与
`src/mandol/core/semantic_map.py`	`SemanticMap`：内容、向持久化
`src/mandol/core/semantic_graph.py`	`SemanticGraph`：rustw关系、邻居与路径
`src/mandol/retrieval/retrieval_interface.py`	`RetrievalMethod`、`RetrievalResult`、`BaseMultiRetrievalInterfa`

路径	作用
`src/mandol/retrieval/query_bundle.py`	`QueryBundle`：一次 qu dense/SPLADE/BM25 特
`src/mandol/retrieval/advance_retriever.py`	`MultiRetriever`：多后 过滤、融合、rerank 编
`src/mandol/retrieval/bm25_retriever.py`	BM25（Best Matching
`src/mandol/retrieval/cosine_retriever.py`	dense/cosine 后端
`src/mandol/retrieval/splade_retriever.py`	SPLADE 稀疏后端
`src/mandol/retrieval/score_fusion.py`	RRF（Reciprocal Rank F 排名融合）、weighted、
`src/mandol/retrieval/rerank_manager.py`	Cross-Encoder reranker
`src/mandol/retrieval/graph_context_expander.py`	图邻居扩展与路径查找
`src/mandol/hierarchical/hierarchical_retriever.py`	L0/L1/L2 分层检索封装
`src/mandol/episodic/episodic_memory_retriever.py`	episodic 事实检索封装
`src/mandol/entity_relation/entity_relation_retriever.py`	entity/relation 检索封装
`src/mandol/triple_retrieval/triple_tower_retriever.py`	三塔路由、塔结果、塔
`src/mandol/memory_router/locomo_tower_router.py`	LoCoMo 类别路由
`src/mandol/memory_router/longmemeval_tower_router.py`	LongMemEval 类别路由
`benchmark_self_host/locomo10/build_graph.py`	自托管图构建入口
`benchmark_self_host/locomo10/retrieve.py`	自托管检索入口
`benchmark_self_host/locomo10/generate.py`	生成入口
`benchmark_self_host/locomo10/score.py`	评分入口
`benchmark_self_host/locomo10/scripts/`	build/retrieve/generate 脚本
`benchmark_locomo/task_eval/`	论文与基准的 LoCoMo
`benchmark_longmemeval/task_eval/`	LongMemEval 评测路径

3.2 核心对象

检索层的数据对象可以分成两类。一类描述记忆本身，包括 `MemoryUnit`、`MemorySpace`、`TowerSpace`、`SemanticMap` 和 `SemanticGraph`；另一类描述一次查询的过程，包括 `QueryBundle`、`RetrievalResult`、各后端 retriever、`MultiRetriever`、`ScoreFusion`、`RerankerManager` 和 `GraphContextExpander`。

3.2.1 MemoryUnit

`MemoryUnit` 是 Mandol 的统一记忆项，字段不多，但把内容和索引串在了一起。

字段	含义
`uid`	跨 `SemanticMap` 与 `SemanticGraph` 的稳定标识
`raw_data`	原始内容，含文本、speaker、session、dialogue_id
`metadata`	层级、记忆类型、来源、时间
`embedding`	dense 向量
`sparse_embedding`	SPLADE 稀疏向量
`text_cached`	检索时使用的文本视图

同一条记忆既能被向量索引命中，也能作为图节点被关系扩展带出来，靠的就是 `uid` 在这两套结构里指向同一个单元。

3.2.2 MemorySpace 与 TowerSpace

`MemorySpace` 提供的是逻辑隔离，不是物理隔离，用来限制一次检索能碰到哪些单元。预实验中出现的空间包括：

```
hierarchical_memory:L0_Observation
hierarchical_memory:L1_Summary
hierarchical_memory:L2_Insight
entity_relation:entities
entity_relation:mentions
entity_relation:relations
episodic_memory
```

`TowerSpace` 把这些空间映射到检索塔，`retrieve.py` 因此可以只检索某一个塔或某个子空间，而不必自己拼空间名。

3.2.3 SemanticMap

`SemanticMap` 是"内容加索引"的集中管理对象，持有 `MemoryUnit` 集合，同时管理 dense 向量与 FAISS（Facebook AI Similarity Search）索引、SPLADE 稀疏向量与倒排索引、BM25 需要的文本、memory space 的成员关系，以及这些索引的持久化、加载和重建。它不负责排序，只负责让各个后端能查到候选。

3.2.4 SemanticGraph

`SemanticGraph` 管结构关系。内部用 `rustworkx.PyDiGraph` 存节点和边，`_uid_to_index` 与 `_index_to_uid` 维护 UID 到图索引的双向映射；`get_explicit_neighbors` 取显式结构邻居，`get_implicit_neighbors` 取语义邻居，另有路径查找和图的加载保存。它与 `SemanticMap` 共享同一批 UID，所以图上的节点可以直接对应回记忆单元。

当前图的边只覆盖 entity 之间的关系。`conv-26` 上是 1916 个节点、254 条边，关系类型有 `friend_of`、`participated_in`、`family_of`、`supports` 等。度分布很稀，p95 为 1，最大度 29，这也是后面图扩展收效有限的结构性原因。

3.2.5 RetrievalResult、BaseRetriever 与 MultiRetrievalInterface

`RetrievalMethod` 枚举了 BM25、cosine、SPLADE、graph traversal、hybrid 等方法；`RetrievalResult` 保存命中的 `unit`、`score`、`method` 和 `method_details`；`BaseRetriever` 规定单个后端要实现 `search()` 和 `get_method_type()`；`MultiRetrievalInterface` 规定多后端协调的接口。这四者构成了后端接入的契约，新增一路检索只要实现 `BaseRetriever` 即可。

3.2.6 QueryBundle

`QueryBundle` 缓存一次 query 的 dense 向量、SPLADE 稀疏向量和 BM25 token。BM25、dense、SPLADE 三个后端都需要 query 特征，如果各自编码一遍就是三倍开销；`QueryBundle` 让它们在一次调用内复用同一份结果。

3.2.7 MultiRetriever

`MultiRetriever` 是检索编排的核心。构造时不加载任何后端，第一次查询才按需实例化 BM25、cosine、SPLADE 和 graph retriever。一次 `smart_search` 依次经过：按 memory space 确定候选范围；并行调用多个后端；用 `ScoreFusion` 做 RRF、加权或平均融合；可选的第一阶段 rerank；可选的图扩展；最后返回 `RetrievalResult` 列表，或者在 `return_detailed=True` 时返回带执行计划的详细字典。

3.2.8 ScoreFusion 与 RerankerManager

`ScoreFusion` 提供三种合并方式。`rrf_fusion` 只看名次：

$$
\mathrm{RRF}(d)=\sum_{m}\frac{1}{k+\mathrm{rank}_m(d)}
$$

`weighted_fusion` 按方法权重合并分数，`average_fusion` 直接对多方法分数取平均。融合结果还可以再交给 Cross-Encoder 重排。`RerankerManager` 负责本地与云端 reranker 的加载、缓存和失败回退，可选 BAAI、Qwen、Jina 等模型。

3.2.9 GraphContextExpander

`GraphContextExpander` 从 seed UID 出发做 1 跳或 2 跳扩展，邻居既取结构也取语义，支持按关系类型过滤和路径查找，结果打包成 `GraphContext`，其中包含 seed、expanded units、relationships 和 paths。预实验中它被单独拿出来做了 P5 探针，探针口径见 5.1 节。

需要注意的是，主链路上 `MultiRetriever` 的 `graph_expander` 字段没有初始化，因此调用 `smart_search(enable_graph_expansion=True)` 并不会真的扩展。这一点在 3.5 节还会

提到。

3.2.10 TripleTowerRetriever 与路由器

`TripleTowerRetriever` 管三塔（hierarchical、graph、episodic）的检索与合并，负责塔结果封装、`tower_separate` 与 `unified_rerank` 两种重排策略，以及路由决策和统计。`LocomoTowerRouter` 按 LoCoMo 的问题类别选择优先塔。

实际运行时要留意一处不一致：自托管的 `benchmark_self_host/locomo10/retrieve.py` 绕过了 `TripleTowerRetriever`，直接调用 `MultiRetriever.smart_search`。论文路径和自托管路径的检索行为因此并不等价，见 3.3.3 节。

对象之间的关系可以用图 2 概括。

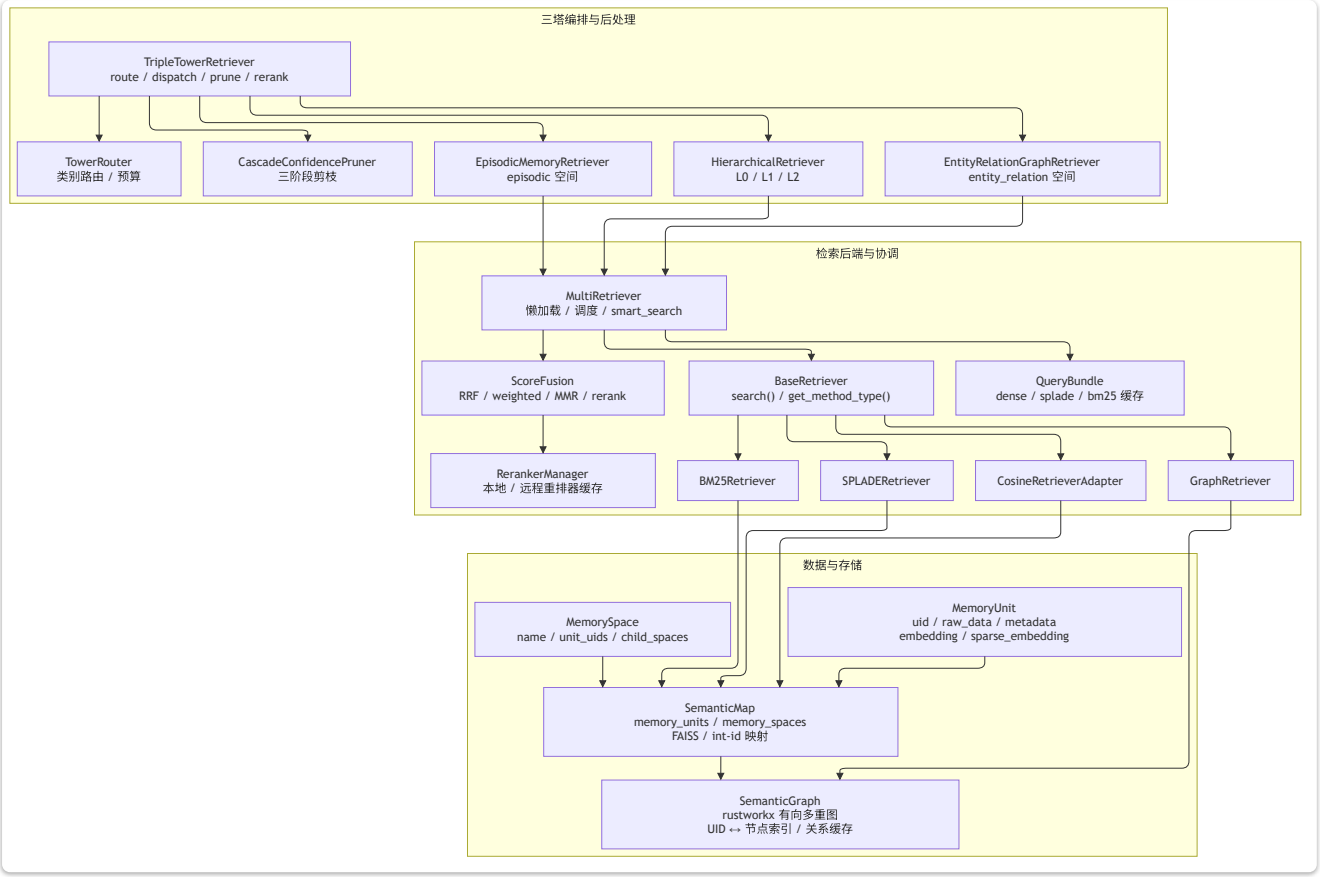


图 2　`MultiRetriever` 与各后端、融合、重排、图扩展及存储对象的关系

阅读这张图时容易忽略一点：`ScoreFusion` 拿到的是各后端返回的 `RetrievalResult`，它并不知道这些结果来自哪个 memory space，也不知道塔的可靠性；空间过滤发生在后端内部，塔的区分发生在更外层。融合层能看到的上下文只有方法名和分数。

3.3 执行流程

3.3.1 离线构建

构建只在实验开始时跑一次，产物落到 `memory/<sample_id>/`：

```
LoCoMo conversation
  → L0 对话单元
  → L1 session 摘要
  → L2 跨 session 洞察
  → episodic 事实
  → entity / mention / relation
  → graph edges
  → BM25 / dense / SPLADE 索引
  → memory/<sample_id>/
```

其中 L1、L2、episodic 和 entity/relation 都是对同一个 `MemoryUnit` 抽象的不同实例化，通过 `metadata` 区分层级与来源，通过 `source_unit_uids` 一路指回 L0 对话。

3.3.2 在线查询

论文路径下，一次查询走的是这条链路：

```
加载 memory/<sample_id>/
  → 选择 tower / memory space
  → 并行 BM25、cosine、SPLADE
  → RRF / weighted fusion
  → 可选 first-stage rerank
  → 可选 tower_separate / unified_rerank
  → final l0 / graph / episodic
  → 下游生成
```

用一个具体问题看：`Who performed at the concert at Melanie's daughter's birthday?`

`Melanie`、`daughter`、`concert` 是实体与关系线索。graph 塔可能返回相关的关系或实体单元，episodic 塔可能返回带日期的事件，L0 塔返回原始对话。三路最后都要落到 `D11:3` 这样的 dialogue ID 上，才算真正拿到证据。这条 query 在预实验中编号为 `CR-R-002`，gold 是 `D11:3`，属于关系加多跳的类型。

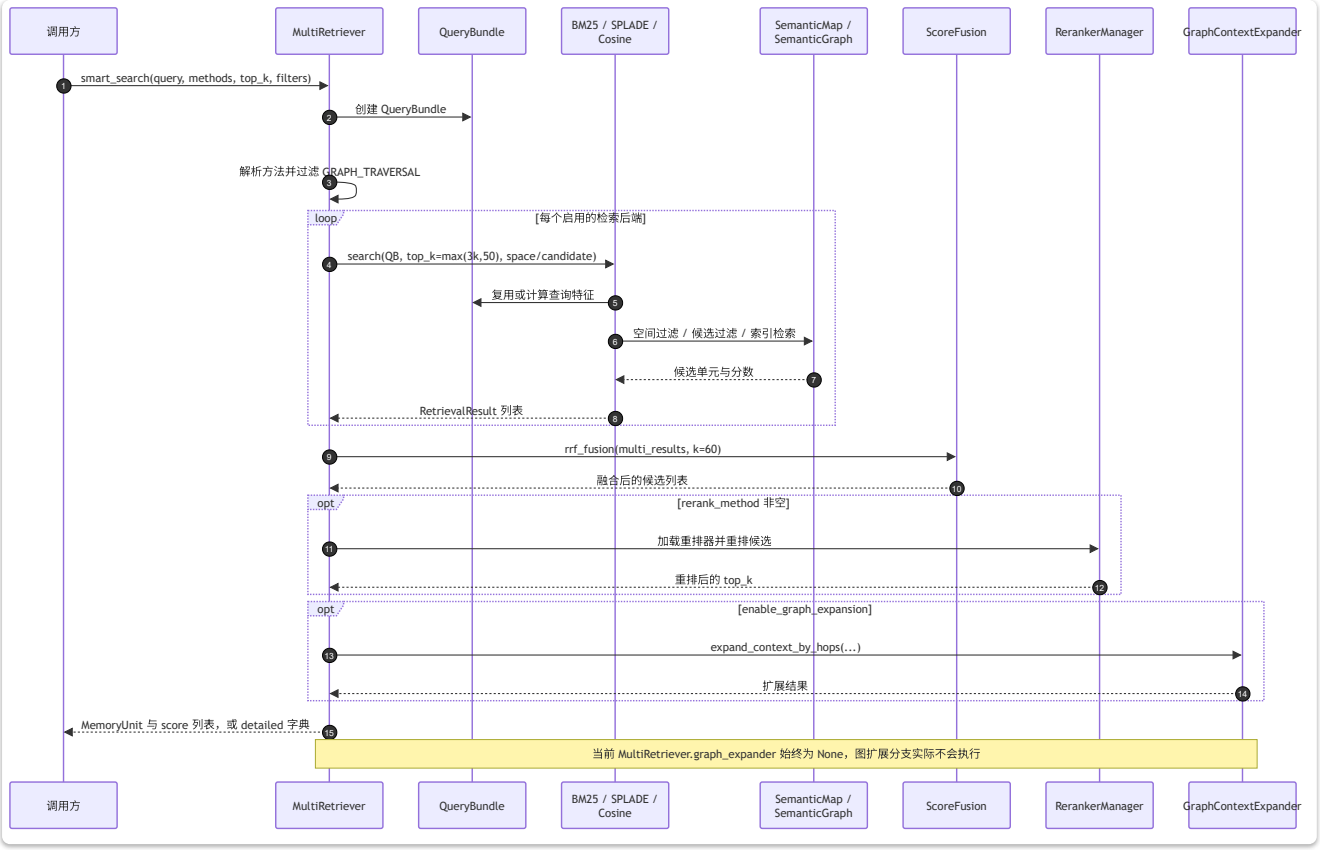


图 3 `smart_search` 的查询执行时序：QueryBundle 复用查询特征，各后端按空间过滤后交给融合与可选重排

图 3 中的 B 到 SM 一段表示空间与候选解析。BM25 和 SPLADE 查的是各自的倒排索引，只有 cosine 真正走 `SemanticMap` 与 FAISS；`QueryBundle` 负责在一次调用内把查询特征算好，给三个后端复用。

3.3.3 自托管脚本实际走的路径

`benchmark_self_host/locomo10/retrieve.py` 的结构与上面这条链路不同：

1. 加载 `SemanticGraph`；
2. 构造 `MultiRetriever`；
3. 对 l0_direct、hierarchical、graph、episodic 分别调用 `smart_search`；
4. 合并 first-stage 结果；
5. 可选执行 second-stage；
6. 写出 `first_stage`、`second_stage`、`final` 三份记录。

它没有经过 `TripleTowerRetriever`，也没有初始化 `graph_expander`。所以自托管跑出来的数字反映的是“四路 `smart_search` 加手工合并”，而不是塔路由加量化剪枝的完整策略。本文后续的预实验都基于这条路径。

3.4 三塔检索与后处理

为了和自托管路径区分，这里单独把 `TripleTowerRetriever` 的流水线记一遍。

路由阶段先判断是否需要动态选塔：`enable_routing=False` 或 `search_mode != ALL_TOWERS` 时直接按配置的塔开关和预算执行；否则按 dataset 选择 `LocomoTowerRouter` 或 `LongMemEvalTowerRouter`，用问题类别得到 `enable_*` 与 `topk_*`，路由异常时回退全塔。三塔分别执行后，`_build_candidates` 把结果拼成 `EnhancedCandidateChunk`，分数做 min-max 归一化，并附上 `timestamp`、`entity_ids`、`memory_space` 和 `tower_source`。接着 `CascadeConfidencePruner` 分三级剪枝：分数硬过滤、跨塔冲突与去重、MMR（Maximal Marginal Relevance，最大边际相关）加 token 预算。最后走 `_unified_rerank`，默认用 BAAI，输出 `fused_context` 和一份量化统计。

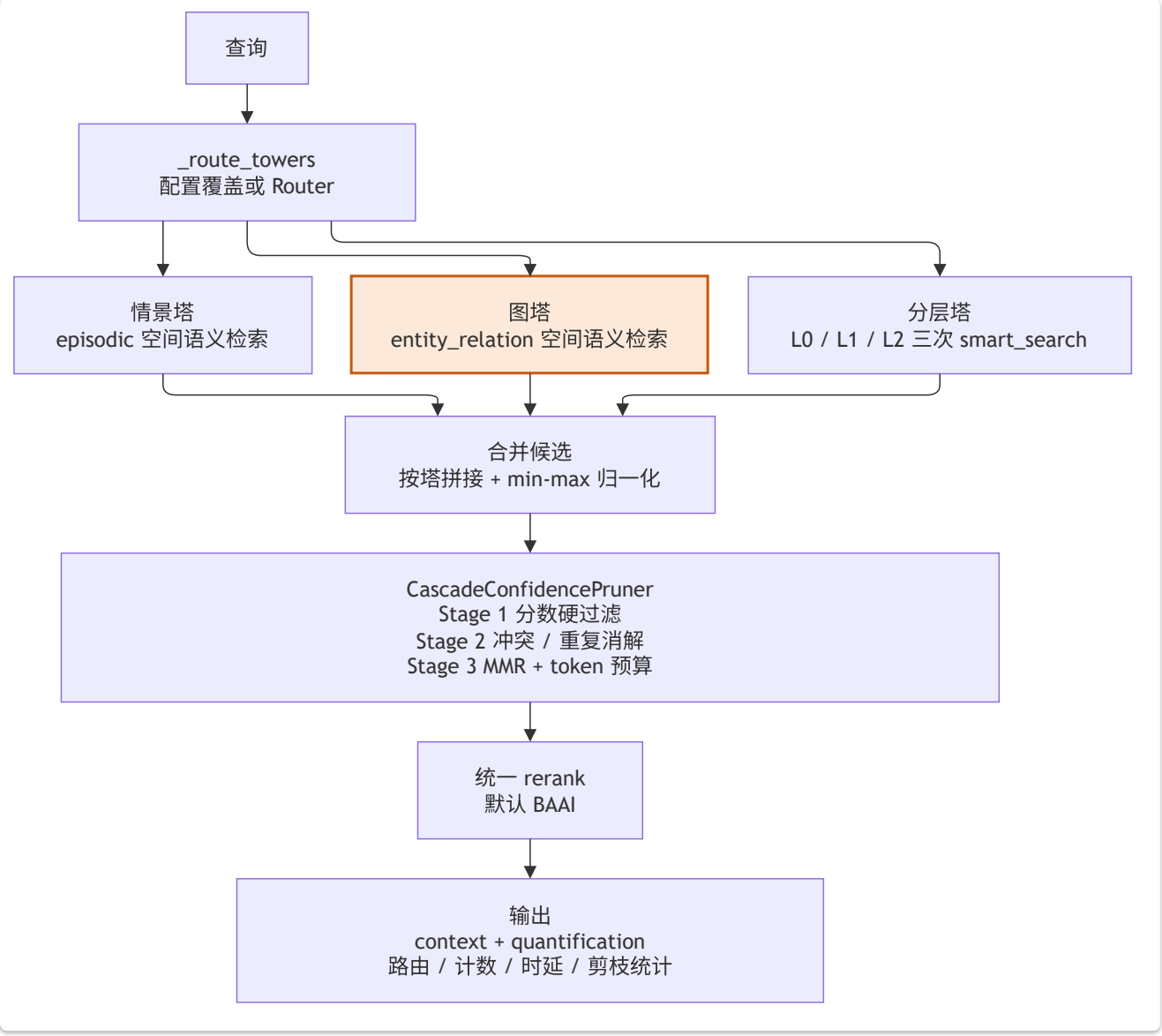


图 4　三塔检索、级联剪枝与统一重排

这条路径有几个容易被忽略的细节。三塔里的"图塔"其实是空间隔离的语义检索，不是图遍历。同一个单元出现在不同塔时，`chunk_id` 会带上塔名前缀，被当成两个候选，只能靠第二级剪枝去重。`final_top_k` 与各塔的 `topk` 是两套预算，剪枝的 `pruner_max_tokens` 又是第三套。路由表是代码里的静态字典，不是在线学习的策略，`LocomoTowerRouter.route()` 主要按类别查表。

3.5 代码层面的限制

预实验和代码阅读中确认了以下几处限制，后续设计要绕开或补齐。

限制	影响
自托管 `retrieve.py` 绕过 `TripleTowerRetriever`	无法直接验证路由器与塔间策略的端到端效果
`MultiRetriever.graph_expander` 未初始化	`enable_graph_expansion=True` 在主链路上是空操作
`EntityRelationGraphRetriever` 是薄封装	`topk_graph`、`use_entity_relation` 在当前路径没有真正生效
融合默认是 RRF 或固定权重	条件类型、实体、时间、关系都没有进入融合特征
证据映射依赖 `source_unit_uids` 等字段	部分 entity 与 relation 只能回退到 session 级别
缺少条件级测试	总体 QA 分数无法区分“条件解析失败”和“排序失败”

---

4 测试与评价代码

4.1 仓库自带的测试

`make test` 只跑两组测试：`tests/test_rocksdb_tiered_cache.py` 覆盖 tiered storage 与 RocksDB 缓存，`examples/mandol_chat/tests/` 覆盖会话 API、健康检查和 benchmark 隔离，合计 5 个测试文件。检索层本身，包括融合、重排、图扩展，没有单元测试。

新增的测试文件要手动登记进 `Mandol/Makefile` 的 `STORAGE_TEST` 或 `TEST_DIR`，否则 `make test` 不会执行到。这个约定在补条件级测试时容易踩坑。

4.2 自托管 LoCoMo10 评测链

`benchmark_self_host/locomo10/REPRODUCE.md` 给出的主流程是：

```
build_graph.py → retrieve.py → generate.py → score.py
```

对应五个脚本：`scripts/run_build.sh`、`run_retrieve.sh`、`run_generate.sh`、`run_score.sh`、`run_all.sh`。每次运行在 `test_runs/<run-id>/` 下生成一套目录：

```
test_runs/<run-id>/
  memory/
  retrieve/
  generate/
  score/
  logs/
```

这条路不经过 router 与 quantification，记忆由 `build_graph.py` 直接从对话构建，检索由 `retrieve.py` 按四路分别调用 `smart_search` 后手工合并，评分在 `score.py` 内完成。它测的是 Mandol 的记忆与检索组件，不是论文里的塔路由策略，这一点在引用数字时必须说明。

4.3 论文与基准评测路径

论文路径的文件包括：

- `benchmark_locomo/task_eval/locomo_triple.py`
- `benchmark_locomo/task_eval/locomo_triple_router.py`
- `benchmark_locomo/task_eval/locomo_triple_router_quantification.py`
- `benchmark_locomo/task_eval/evaluation.py`
- `benchmark_longmemeval/task_eval/benchmark_triple.py`
- `benchmark_longmemeval/task_eval/evaluation.py`

这些脚本覆盖了 triple/tower 检索、router 加 quantification、生成与 LLM judge、QPS（每秒查询数）与延迟成本，数据集有 LoCoMo 和 LongMemEval 两个。它们适合做最终对比，但不适合调试单个条件，因为一次运行要串起整条链路。

4.4 本研究使用的指标

层面	指标
检索	Evidence Recall@k、hit@k、MRR（Mean Reciprocal Rank，平均倒数排名）、nDCG（Normalized Discounted Cumulative Gain，归一化折损累计增益）
塔	unique contribution（单个塔的独有贡献）、Oracle 子集（事后最优的塔组合）、regret（相对 Oracle 的差距）
条件	gold 保留率、噪声下降、FAR（False Accept Rate，错误接受率）、Abstention Accuracy（正确拒答率）
生成	EM（Exact Match，精确匹配）、F1、LLM judge
成本	延迟、token、API 调用次数、构建时间

指标口径统一以 L0 的 `dialogue_id` 为锚点，L1/L2/episodic/entity/relation 通过 `source_unit_uids` 递归解析到 L0；解析不到的单元单独统计，不进分母。

4.5 现有评测的缺口

现有评测能回答"整体答对了多少"，回答不了"哪个条件没有落地"。缺口集中在四处：没有按实体、时间、关系、否定、状态分层的测试集；没有 BM25、dense、SPLADE 的方法级消融，看不出单一后端的贡献；自托管路径绕过 router，塔策略无法端到端验证；证据映射只到 session 级，指标容易偏高。因此正式实验要同时补条件级测试和方法级消融，只靠 LoCoMo 的总体分数不够。

---

5 预实验结果与问题诊断

5.1 实验口径

预实验在 exp0.1 阶段完成，共 30 条 probe。probe 是人工标注的检索探针，一条 query 带上条件、gold 答案和证据对话编号。按能力族分，有时间与逻辑、实体关系、多跳聚合、否定与错误实体、状态更新、别名与指代六类；按所需数据分三档，Tier A 只用 `conv-26`，Tier B 需要 `conv-30`，Tier C 需要其他样本图。真正进入主结果的是 Tier A 的 15 条，其中 11 条正样本、4 条负样本。Tier B 的 2 条依赖 `conv-30` 的部分图，Tier C 的 13 条依赖尚未构建的样本图，都只做了局部检查。

探针	内容	主要产出
P4	证据映射与检索指标	`results/metrics.json`、`retrieval_metrics*.csv`
P5	图扩展离线探针，32 组配置 × seed top-1/3	`graph_expansion*.csv`
P6	硬约束、软约束、FAR 与 abstention	`constraint_probe*.csv`
P7	塔子集与权重的 Oracle 上限、可预测性	`policy_probe.json`
P8	小样本生成与评分	`answer_subset.csv`

表中的产物路径相对 `OOP_docs/exp0.1/`。P1 到 P3 分别是图构建、probe 生成与检索矩阵，属于准备和数据采集，没有列进来；进入分析的是 P4 到 P8。

运行口径如下。图与检索来自 run `exp0.1_20261021_132255_deepseek`；构建、生成和 judge 使用 `deepseek-v4.1-flash`，嵌入用 `Qwen/Qwen3-Embedding-0.6B`，稀疏检索用 `naver/splade-v3`。`baseline_direct` 是关闭 first-stage 与 second-stage rerank 的代理基线，默认 baseline 因 BAAI reranker 在 WSL 下触发 `torch._inductor` 编译失败而超时，`unified_rerank` 因此未测。P5、P6、P7 都是离线分析，没有修改 `Mandol/` 的生产代码。

规模与基础数据：图 1916 节点、254 条边；检索两套配置均为 199/199 成功；证据映射 1916 个单元中 1912 个解析成功，4 个未解析，未解析率 0.21%；3 条已知 probe 的 gold evidence 全部映射成功并人工核对过 L0 原文。

5.2 检索：候选饱和，排序是瓶颈

Tier A 的 11 条正样本结果如下。

指标	baseline_direct	high_recall
first-stage union Recall@10	1.000	1.000
final union Recall@10	0.765	0.765
final union hit@10	1.000	1.000
final union MRR@10	0.482	0.486
final union nDCG@10	0.498	0.500
final union Recall（全部）	1.000	1.000

两套配置的 first-stage union Recall@10 都是 1.000，说明候选阶段已经把 gold 捞齐。把 first-stage 预算从 15/15/30/30 提到 50/50/100/100（依次为 l0、hierarchical、graph、episodic）之后，final Recall@10 仍然是 0.765，扩大的候选只落在长尾上，没有进入最终 top-10。hit@10 是 1.000 而 Recall@10 只有 0.765，差的部分来自需要多条证据的 probe：至少找到一条，但没找全。nDCG@10 停在 0.50 附近，说明排序还有明显的改进空间。

按能力族拆分时，聚合类的 final Recall@10 是 0.583，反事实和否定各 0.500，关系与多跳是 0.833；但每条证据在“全部”口径下都能覆盖。问题因此不在候选是否存在，而在 top-k 里排不排得进去。

5.3 图扩展：没有增量

P5 在 `high_recall` 的 graph top-100 上取 top-3 作为 seed，比较纯结构邻居与 `GraphContextExpander`（结构加语义）。

配置	相对 seed 的 gold 增益	相对 graph 的增量	噪声	扩展单元	延迟
structural 1-hop, 10	0.068	0.000	0.811	4.8	0.02 ms
expander 1-hop, 10	0.485	0.000	0.787	12.8	7.6 ms
structural 2-hop, 10	0.068	0.000	0.809	20.0	0.08 ms

配置	相对 seed 的 gold 增益	相对 graph 的增量	噪声	扩展单元	延迟
expander 2-hop, 10	0.561	0.000	0.842	62.2	15.6 ms

“相对 seed 的 gold 增益”从 0.068 到 0.561，看上去语义扩展很有用；但“相对 graph 的增量”全部是 0.000。原因是 graph top-100 已经覆盖了这些 dialogue，扩展出来的单元没有带来新证据。代价却不小：2 跳语义扩展每次带出 62.2 个单元，噪声 0.842，其中约 45% 是 hub 节点，也就是度数很高、连接大量邻居的实体。纯结构扩展几乎没有收益，图本身太稀。

另有一处弱信号：在 `baseline_direct` 的 graph top-30 上，2 跳 expander 相对 graph 多覆盖约 6.2 个百分点，但只覆盖 8 条有 seed 的正样本，噪声约 0.78，不足以支持把它接入生产链路。扩展路径本身的可追溯比例是 1.0，每条扩展都能查回 seed。

5.4 硬约束与软约束：精确实体过滤已经在丢 gold

P6 用 probe 标注的实体集合模拟硬过滤，再和软打分对比，结果都基于 `high_recall` 的 final candidates。

方法	Recall@10	gold 保留率	噪声下降	FAR@10	Abstention
baseline	0.697	1.000	0.000	0.750	—
实体过滤 lenient	0.697	1.000	0.050	0.500	—
实体过滤 token strict	0.697	1.000	0.178	0.500	—
实体过滤 exact	0.591	0.848	0.222	0.500	—
soft LR	0.629	1.000	0.000	1.000	0.500
soft MLP	0.598	1.000	0.000	1.000	0.216
strict + LR	0.667	1.000	0.178	0.750	—

宽松匹配和 token 级严格匹配都不丢 gold，后者还能削掉 17.8% 的候选噪声，但两者的 Recall@10 都停在 0.697。噪声少了，排序没有变好，这说明噪声不是 top-k 失败的主因。精确匹配则直接丢 gold：保留率 0.848，Recall@10 掉到 0.591，换算下来 15.2% 的 gold 被过滤掉，低于 90% 的验收线。

软约束也没能翻盘。LR 和 MLP 的 Recall@10 都低于 baseline，FAR@10 反而是 1.000，等于来者不拒；LR 的 balanced abstention 只有 0.50，MLP 是 0.216。

时间硬过滤没有测成。Tier A 里 5 条带时间条件的 probe（按 `conditions.time` 字段统计）用的都是 `recently`、`used to`、`after_event`、`counterfactual` 这类相对、事件或反事实表述，没有可比的绝对区间；Tier C 有 7 条显式日期区间，但对应图还没建。

5.5 策略上限：Oracle 增益小且不可预测

P7 枚举了三个 final 塔的 7 个非空子集，再试加权融合。

指标	值
全塔 Recall@10	0.697
Oracle 子集 Recall@10	0.720
Oracle 子集增益	+2.3 个百分点
Oracle 权重 Recall@10	0.735
Oracle 权重增益	+3.8 个百分点
全局权重 5-fold CV	0.689
查询特征预测子集 CV	0.424
子集预测准确率 / majority	0.455 / 0.273

Oracle 子集把 Recall@10 从 0.697 提到 0.720，加权融合到 0.735，上限不高但确实存在。问题是这个上限拿不到：全局最优权重做 5 折交叉验证只有 0.689，比 full 还低；用查询特征预测子集，准确率 0.455 对 majority 0.273 看似有信号，但预测后执行得到的 Recall@10 只有 0.424。每 query 的策略计算只要约 3 ms，成本不是障碍，可预测性才是。

5.6 生成层：小样本下仍有明显错误

P8 在 15 条 Tier A 上跑了两套可用上下文，生成模型、prompt 和 judge 固定。

指标	baseline_direct	high_recall
EM（11 条 answer）	0.091	0.000
F1（11 条 answer）	0.284	0.271
LLM judge accuracy（15 条）	0.733	0.800
Abstention Accuracy（4 条 abstain）	0.750	0.750
正样本误拒答率	0.182	0.182

EM 在自然语言答案上几乎不可用，0.091 和 0.000 都不能说明检索好坏；F1 也只能算持平。judge 准确率是 0.733 和 0.800，abstention 都是 0.750，但 11 条正样本里仍有 18.2% 被错误拒答。

错误分三类。证据已经在上下文里、模型却拒答，例如 T-001 和 T-002；负样本里出现错误实体的证据，模型照着答了，例如 N-001；答案改写与 gold 字面不一致，被 EM 判错。`unified_rerank` 未测，所以这里只是两套上下文的对比，不是三套。

5.7 问题诊断

把五个探针放在一起，短板集中在三处。

第一处是条件到证据的落地。精确实体过滤丢 15.2% gold，图扩展的噪声也主要来自 session 级映射：502 个 entity 里有 464 个是 `merged_*`，多数只能回退到 `session_ids`，证据粒度停在上下文级而不是 turn 级。条件解析得再准，落不到具体对话上也起不到作用。

第二处是排序。候选覆盖已经饱和，差距全在 top-k 选择上。现有融合是固定 RRF 或固定权重，既没有用条件，也没有用塔的可靠性；P6 的软打分和 P7 的学习型策略都没能把覆盖转成稳定收益。

第三处是评测。Tier A 只有 15 条，单样本，显式时间缺失，多跳路径没有核验，第二人复核也还没做。这些数字只能支持方向性判断。

---

6 初步设计方案

6.1 与现有 Mandol 设计的差异

现有主链路是：

```
Query
→ MultiRetriever.smart_search
→ BM25 / cosine / SPLADE
→ RRF 融合
→ 可选 rerank
→ final 10 / graph / episodic
```

这条链路的融合是检索信号融合，不是条件感知融合。条件解析、证据 grounding、条件级评测都不在现有主链路里。

拟议方案在入口和出口各加一层，中间改造融合与重排：

```
Query
→ ConditionParser
→ EvidenceGrounder
→ 条件感知 MultiRetriever
→ ConditionAwareFusion
→ ConditionAwareReranker
→ EvidenceValidator
→ final
```

两者的差异按层次列在下表。

层次	原 Mandol	新增或修改	建议位置
查询理解	`QueryBundle` 与类别路由	`ConditionSet` / `ConditionParser`	新增 `retrieval/condition`
条件 grounding	没有条件到记忆单元的显式映射	`EvidenceGrounder` / `EvidenceLink`	新增 `retrieval/evidence_`
索引层	BM25、dense、SPLADE、FAISS、图索引	实体别名、时间区间、关系路径、状态版本等轻量条件索引	在 `SemanticMap` / `Semanti` 元数据或二级索引
候选生成	各塔统一 `smart_search`，按空间过滤	`smart_search` 增加可选的 `ConditionSet` / `FusionPolicy`	`retrieval/advance_retri`
方法级融合	BM25/cosine/SPLADE 固定 RRF	条件感知的方法权重	扩展 `retrieval/score_fus` `condition_aware_fusion.`
塔级融合	`tower_separate` / `unified_rerank`	条件特征与塔门控	`triple_retrieval/triple_` 或新融合层
证据重排	主要依赖 Cross-Encoder	条件匹配、证据覆盖、去重、来源一致性	新增 `ConditionAwareRera`
图扩展	`GraphContextExpander` 存在但未接入	只在关系或多跳且置信度高时启用	`MultiRetriever` 初始化与
证据校验	没有统一的条件证据校验	`EvidenceValidator`	融合后、生成前
评测	QA 级分数	条件级测试、方法级消融、FAR 与 abstention、成本	`benchmark_self_host` 与分

6.2 第一层：条件解析与证据 grounding

6.2.1 条件对象模型

对象	字段	说明
`ConditionSet`	entities, time, relation, negation, state, speaker, session, confidence	一次查询的条件集合
`Condition`	type, raw_text, normalized_value, hard_or_soft, source_span, confidence	单条条件，保留原文片段便于回溯
`EvidenceLink`	unit_uid, dialogue_id, source_path, granularity, unresolved_reason	条件落到哪个证据、粒度多粗、为什么没落上

一个 `ConditionSet` 包含若干 `Condition`，每条 `Condition` 通过若干 `EvidenceLink` 指向 `MemoryUnit`，`MemoryUnit` 再被检索结果引用。这样"条件是否被满足"可以在证据层判断，而不必等生成完再看答案。

6.2.2 需要解析的内容

条件	解析内容
实体	规范名、别名、指代、speaker
时间	绝对日期、区间、相对时间、事件顺序
关系	类型、方向、跳数、路径
否定	否定词、反事实、错误实体
状态	旧状态、新状态、有效时间
speaker/session	对话参与者、会话边界

6.2.3 证据 grounding

条件要沿着现有字段一路落到 L0：

```
Condition
→ candidate entities / time intervals
→ MemoryUnit
→ source_unit_uids / source_l1_uids
→ L0 dialogue_id
```

这里有几条硬要求。每条证据必须记录粒度，是 turn、session、summary 还是 relation，别让 session 级命中冒充 turn 级。对 `unknown` 或 session-level 的回退要显式标记，不能悄悄当成精确命中。`unresolved` 的单元要单独计数，不能静默丢弃。验收线是精确实体过滤的 gold 保留率不低于 90%，达不到就先修解析，不进硬过滤。

6.2.4 新增索引

建议加轻量二级索引，而不是重写存储。

索引	用途
EntityAliasIndex	规范名、别名、指代到实体
TimeIntervalIndex	绝对日期或区间到记忆单元
RelationPathIndex	关系类型、方向、路径
StateVersionIndex	状态更新与有效时间

这些索引可以挂在 `SemanticMap` 与 `SemanticGraph` 的元数据层，`MemoryUnit` 的序列化格式不用动，老数据也能直接加载。

6.3 第二层：条件感知的融合与重排

6.3.1 方法级融合

当前 `ScoreFusion.rrf_fusion` 只看名次。拟增加条件感知的权重：

$$
S(d)=\sum_{m} w_m(c),s_m(d)+\lambda,\mathrm{CondMatch}(d,c)
$$

其中 m 取 BM25、dense、SPLADE，$w_m(c)$ 由条件类型和置信度决定，$\mathrm{CondMatch}$ 来自实体、时间、关系、否定的匹配结果。这样设计的假设是三类条件对后端的偏好不同：时间和精确实体更依赖词面与稀疏信号，语义改写更依赖 dense，关系与多跳更依赖图。这个假设本身也要在 E2 里检验。

6.3.2 塔级融合

塔级现在只有 `tower_separate` 与 `unified_rerank` 两种策略，没有条件输入。拟让权重随条件变化：关系和多跳条件提高 graph 塔权重，时间与事件条件提高 episodic 塔权重，语义聚合类问题提高 hierarchical 与 L0 的权重。低置信条件只做软信号，不触发硬过滤。

6.3.3 条件感知重排

在 Cross-Encoder 之后加一层轻量选择，特征包括条件匹配、证据覆盖度、证据去重、来源一致性、时间与关系一致性。目标是把 top-k 的 Recall@10 和 nDCG 提上去，而不是继续堆候选。选择这一层而不是直接上 MLP 的原因很直接：P7 已经说明学习型路由在当前特征下预测不出来，而这几个特征都可以单独消融、单独解释。

6.3.4 硬约束的使用边界

P6 已经证明精确实体过滤会丢 15.2% 的 gold，所以设计上默认不做硬过滤。只有条件 grounding 置信度高、实体或时间已经精确解析时才启用；低置信条件退为软特征；一旦过滤后候选为空或校验失败，就回退到原 `MultiRetriever` 的结果。硬过滤是加速手段，不是正确性保证。

6.4 第三层：端到端验证

6.4.1 实验矩阵

实验	目的	对照
E1 方法级消融	BM25、dense、SPLADE 各自的贡献	单方法 vs RRF
E2 条件感知融合	条件特征是否提升 Recall@10 与 nDCG	原 RRF vs 条件融合
E3 塔级融合	条件门控是否优于全局权重	`tower_separate` vs 条件塔融合
E4 硬约束与软约束	条件 grounding 修好后重测	lenient / strict / exact
E5 端到端生成	检索收益是否转化为答案收益	F1、judge、abstention
E6 成本	延迟、token、索引维护	原链路 vs 新链路

6.4.2 数据覆盖

Tier A 只有 15 条，且全在 `conv-26`，不足以支撑正式结论。正式实验需要补齐：完整构建 `conv-30` 的 L1/L2 与 SPLADE；构建 Tier C 的 7 个样本图；补充显式日期区间、多跳路径、状态更新、别名与指代四类样本；把 probe 扩到约 60 条；安排 20% 到 30% 的第二人复核。probe v0.1 冻结后不再根据测试结果修改，调参一律在独立 dev 集上做。

6.4.3 预期收益与停止条件

方向	预期	依据
条件 grounding	exact 过滤的 gold 保留率从 0.848 提到 0.90 以上	P6
方法级融合消融	未知，必须先测	当前没有拆方法
条件感知融合	保守目标 +2 到 +4 点 Recall@10	P7 的 Oracle 只有 +2.3 与 +3.8，且预测器未实现
图扩展	当前约 0 增量	P5
硬过滤	不作为默认路径	P6 的 exact 丢 15.2% gold
MLP 路由	暂不训练	P7 的查询特征预测失败

停止条件尽量写成可判定的形式：

- 条件 grounding 修好后，exact 过滤仍低于 90%，停止硬约束方向；
- 条件感知融合在独立 dev 上没有超过 2 个点的 Recall@10 或 nDCG 提升，停止该方向；
- 图扩展在关系子集上不能超过 graph 基线 3 个点，停止图扩展；
- Oracle 路由增益低于 1 到 2 个点，或查询特征不可预测，停止 MLP。

6.5 已否决的方案及其原因

方案迭代的过程需要留痕。预实验之后，下面几条方向被否掉，理由和重开条件一并记下。

方案	否决依据	重新考虑的条件
把图扩展接进主链路	P5 相对 graph 增量 0.000，2 跳噪声 0.842，hub 占 45%	关系类子集上超过 graph 基线 3 个点
精确实体硬过滤	P6 gold 保留率 0.848，Recall@10 从 0.697 掉到 0.591	实体链接与别名解析修好后保留率不低于 0.90
软 LR 或 MLP 打分	P6 无增益，FAR@10 为 1.000，abstention 只有 0.50 与 0.216	有独立 dev 集且 FAR 可控
MLP 或学习型路由	P7 预测后 0.424，低于 full 0.697，全局权重 CV 也只有 0.689	Oracle 增益不低于 2 个点且特征可预测
时间硬过滤	Tier A 没有可比的绝对区间，未测	Tier C 的显式区间建图之后
扩 top-k 提召回	候选已经饱和，Recall@10 没有变化	出现候选确实不足的能力族

这些结论都带条件：现有证据只支持到这一步。每条都留了重新考虑的门槛，前置问题修好之后可以按门槛重开。

---

7 与现有代码的衔接方案

接入方式以不破坏现有接口为前提，分四步落地。

第一步是保持现有接口。`MultiRetriever.smart_search` 的原参数继续可用，`condition_set` 与 `fusion_policy` 作为可选参数加入；旧 benchmark 不传新参数时行为不变，回归测试才能直接复用。

第二步是新增实验模块。解析、grounding、融合、重排各自独立成文件，互不牵连：

```
retrieval/condition_parser.py
retrieval/evidence_grounding.py
retrieval/condition_aware_fusion.py
retrieval/condition_aware_reranker.py
benchmark_self_host/locomo10/condition_eval.py
```

第三步是复用现有后端。BM25、cosine、SPLADE、FAISS、rustworkx 都不替换；条件索引以元数据或二级索引的形式接入；图扩展复用 `GraphContextExpander`，不另写一套。

最后一步是控制开关与不变量。新增 `enable_condition_parsing`、`enable_condition_fusion`、`enable_condition_rerank`、`enable_graph_expansion`、`enable_hard_filter` 五个开关，后两个默认关闭。`MemoryUnit` 的核心字段、`SemanticMap` 与 `SemanticGraph` 的持久化格式、`retrieve.py` 的默认行为、论文 benchmark 的默认基线都不改。

---

8 风险与替代方案

风险	影响	应对
条件解析错误	硬过滤误杀 gold	默认软约束，保留 unresolved，小步验证
数据覆盖不足	结论不可外推	补 Tier B/C、显式时间、多跳路径
reranker 不稳定	无法比较默认 baseline	先修 `torch._inductor` 或换稳定后端
证据粒度粗	指标虚高	记录 granularity，做严格与宽松两套指标
过拟合	在 15 条 probe 上调参	冻结 v0.1，独立 dev，按 sample 切分
API 成本	构建与生成昂贵	缓存图，小样本 smoke，离线分析优先
维护成本	新索引与新模块过多	先做轻量元数据索引，不重写存储

如果条件解析投入过大，可以先做方法级融合消融和证据级重排，把条件解析留到下一阶段。如果 `unified_rerank` 始终跑不稳定，就先固定 `tower_separate` 作为基线，并在报告里注明差异。如果 Tier C 建图成本过高，就先扩 Tier A 的 probe 数量，但结论必须写明只适用于单样本。

---

9 结论

Mandol 的检索层已经具备多后端、多塔和图结构的基础。当前瓶颈不在候选数量，而在条件没有被表示、也没有落到证据上，融合和排序没有条件感知。P4 到 P7 的四个检

索探针从不同角度指向同一个位置：候选覆盖饱和，排序与 grounding 不足，图扩展和硬约束在现有数据上都不划算。

据此，后续工作按下面的顺序推进。先修实体链接、别名与证据粒度，把 exact 过滤的 gold 保留率做到 90% 以上；再做方法级融合消融，确认 BM25、dense、SPLADE 各自的贡献；然后接入条件感知的融合与重排，用条件级指标和端到端指标验证收益；图扩展、硬过滤和 MLP 路由降为可选或后续工作，各自的重新评估条件写在 6.5 节。

预期收益是保守的 +2 到 +4 个点 Recall@10，这是目标，不是已经实现的结果。`unified_rerank`、绝对时间硬过滤、P8 的完整样本和 Tier B/C 仍未测，本文在相关位置逐项标注；在这些缺口补齐之前，任何更大的数字都缺依据。

参考资料

1. Mandol 源代码与官方文档：https://github.com/AgentCombo/Mandol，以及仓库内 `Mandol/src/mandol/`、`Mandol/docs/current/`、`Mandol/benchmark_self_host/locomo10/REPRODUCE.md`。
2. Maharana A., Lee D.-H., Tulyakov S., Bansal M., Barbieri F., Fang Y. Evaluating Very Long-Term Conversational Memory of LLM Agents. ACL 2024: 13851–13870. DOI: 10.18653/v1/2024.acl-long.747.（LoCoMo 数据集）
3. Cormack G. V., Clarke C. L. A., Büttcher S. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. SIGIR 2009: 758–759. DOI: 10.1145/1571941.1572114.（RRF）
4. Formal T., Piwowarski B., Clinchant S. SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking. SIGIR 2021: 2288–2292. DOI: 10.1145/3404835.3463098.
5. Johnson J., Douze M., Jégou H. Billion-Scale Similarity Search with GPUs. IEEE Transactions on Big Data, 2021, 7(3): 535–547. DOI: 10.1109/TBDATA.2019.2921572.（FAISS）